

1 Segment Tree

1.1 Normal

```
1 // Indexada em 0, [a, b]
2 class Segtree{
3     private:
4         ll N;
5         vector<ll> ts;
6     public:
7         // o 0 eh elemento neutro da soma
8         Segtree(size_t n) : N(n), ts(4*n, 0) {}
9         ll query(ll ql, ll qr){
10             return query_range(ql, qr, 0, N-1, 1);
11         }
12         void update(ll i, ll v){
13             return update_range(i, v, 0, N-1, 1);
14         }
15     private:
16         ll query_range(ll ql, ll qr, ll l, ll r, ll no){
17             if(l >= ql && r <= qr) return ts[no];
18             if(l > qr || r < ql) return 0;
19             ll mid = (l+r)/2;
20             ll esq = query_range(ql, qr, l, mid, 2*no);
21             ll dir = query_range(ql, qr, mid+1, r, 2*no+1);
22             return merge(esq, dir);
23         }
24         void update_range(ll i, ll v, ll l, ll r, ll no){
25             if(l==r){
26                 ts[no] = v;
27                 return;
28             }
29             ll mid = (l+r)/2;
30             if(i<=mid) update_range(i, v, l, mid, 2*no);
31             else update_range(i, v, mid+1, r, 2*no+1);
32             ts[no] = merge(ts[2*no], ts[2*no+1]);
33         }
34         ll merge(ll a, ll b){
35             return a+b;
36         }
37     };
```

1.2 Lazy Propagation

```
1 template<typename T> class SegmentTree {
2     private:
3         int N;
4         vector<T> ns, lazy;
```

```

5 public:
6     SegmentTree(const vector<T>& xs)
7         : N(xs.size()), ns(4*N, 0), lazy(4*N, 0) {
8         for(size_t i = 0; i < xs.size(); i++)
9             update(i, i, xs[i]);
10    }
11    void update(int a, int b, T value) {
12        update(1, 0, N-1, a, b, value);
13    }
14 private:
15    void update(int node, int L, int R, int a, int b, T
16        value) {
17        if(lazy[node]) {
18            ns[node] += (R - L + 1) * lazy[node];
19            if(L < R) {
20                lazy[2*node] += lazy[node];
21                lazy[2*node + 1] += lazy[node];
22            }
23            lazy[node] = 0;
24        }
25        if(a > R || b < L)
26            return;
27        if(a <= L && b >= R) {
28            ns[node] += (R - L + 1) * value;
29            if(L < R) {
30                lazy[2*node] += value;
31                lazy[2*node + 1] += value;
32            }
33            return;
34        }
35        update(2*node, L, (L + R)/2, a, b, value);
36        update(2*node + 1, (L + R)/2 + 1, R, a, b, value);
37        ns[node] = ns[2*node] + ns[2*node + 1];
38    }
39 public:
40    T RSQ(int a, int b) {
41        return RSQ(1, 0, N - 1, a, b);
42    }
43 private:
44    T RSQ(int node, int L, int R, int a, int b) {
45        if(lazy[node]) {
46            ns[node] += (R - L + 1) * lazy[node];
47            if(L < R) {
48                lazy[2*node] += lazy[node];
49                lazy[2*node + 1] += lazy[node];
50            }
51            lazy[node] = 0;
52        }
53        if(a > R || b < L)

```

```

53         return 0;
54         if(a <= L and b >= R)
55             return ns[node];
56         T x = RSQ(2*node, L, (L + R)/2, a, b);
57         T y = RSQ(2*node + 1, (L + R)/2 + 1, R, a, b);
58         return x + y;
59     }
60 };

```

2 Algoritmo de Kahn

```

1 // Na construçao da entrada, faça in[x].insert(y), out[y].
  insert(x)
2 vll kahn(ll n, set<ll> in[], set<ll> out[]) {
3     vll o;
4     priority_queue<ll, vll, greater<ll>> q;
5     rep(i, 0, n) {
6         if(in[i].empty())
7             q.push(i);
8     }
9     while(!q.empty()) {
10         ll u = q.top(); q.pop();
11         for(auto e: out[u]) {
12             in[e].erase(u);
13             if(in[e].empty())
14                 q.push(e);
15         }
16         o.push_back(u);
17     }
18     return (o.size()==n ? o : vll()); // Retorna vazio, caso
    haja ciclo
19 }

```

3 Verificar se o grafo é bipartido

3.1 BFS

```

1 // as cores sao 0 e 1
2 bool isBipartite(ll s, ll n, const vvll &adj) {
3     queue<ll> q;
4     q.push(s);
5     vll color(n, -1); color[s]=0;
6     bool flag = true;
7     while (!q.empty())
8     {

```

```

9         for(auto nex: adj[q.front()]) {
10             if(color[nex] == -1) {
11                 color[nex] = 1-(color[q.front()]);
12                 q.push(nex);
13             }
14             else if(color[nex] == color[q.front()]) {
15                 flag = false;
16                 break;
17             }
18         }
19         q.pop();
20     }
21     return flag;
22 }

```

3.2 DFS

```

1 bool dfs(ll u, vll adj[], vll &color, ll c, ll &count0, ll &
count1){
2     vis[u] = true;
3     color[u] = c;
4     if(c==0) count0++;
5     else count1++;
6     for(auto &v: adj[u]){
7         if(!vis[v]){
8             if(!dfs(v, adj, color, c^1, count0, count1)) return
false;
9         }else{
10             if(color[u] == color[v]) return false;
11         }
12     }
13     return true;
14 }

```

4 Dijkstra

```

1 /**
2  * @param g Graph (w, v).
3  * @param s Starting vertex.
4  * @return Vectors with smallest distances from every
5  * vertex to s and the paths.
6  * Na construçao do grafo, nos pares, primeiro vem o peso
7  * e depois o vertice
8  */
9 pair<vll, vll> dijkstra(const vvp11& g, ll s) {
    vll ds(g.size(), LLONG_MAX), pre = ds;

```

```

10 priority_queue<pll, vpll, greater<>> pq;
11 ds[s] = 0, pq.emplace(ds[s], s);
12 while (!pq.empty()) {
13     auto [t, u] = pq.top(); pq.pop();
14     if (t > ds[u]) continue;
15     for (auto [w, v] : g[u])
16         if (t + w < ds[v]) { // relaxamento
17             ds[v] = t + w, pre[v] = u;
18             pq.emplace(ds[v], v);
19         }
20 }
21 return { ds, pre };
22 }
23 vll get_path(const vll& pre, ll u) {
24     vll p;
25     while (u != LLONG_MAX) {
26         p.eb(u), u = pre[u];
27     }
28     reverse(all(p));
29     return p;
30 }

```

5 Kruskal (MST)

```

1 #define tll    tuple<ll, ll, ll>
2 #define vtll   vector<tll>
3 struct DSU {
4     vll parent, size;
5     DSU(ll sz) : parent(sz), size(sz, 1) { iota(all(parent),
6         0); }
7
8     ll find(ll x) {
9         assert(0 <= x && x < parent.size());
10        return parent[x] == x ? x : parent[x] = find(parent[
11            x]);
12    }
13
14    void merge(ll x, ll y) {
15        ll a = find(x), b = find(y);
16        if (size[a] > size[b]) swap(a, b);
17        parent[a] = b;
18        if (a != b) size[b] += size[a], size[a] = 0;
19    }
20
21    bool same(ll x, ll y) { return find(x) == find(y); }
22 };
23 vtll kruskal(vtll& edges, ll n) {
24     DSU dsu(n);
25     vtll mst;
26     sort(all(edges)); // change order if want maximum

```

```
23     for (auto [w, u, v] : edges) if (!dsu.same(u, v)) {  
24         dsu.merge(u, v);  
25         mst.eb(w, u, v);  
26     }  
27     return mst;  
28 }
```